Programação Funcional

Prática a07 — Revisão para a frequência

Fundamentos, tipos, listas e recursão

Revisão · escolha múltipla · escrita de código

O que vamos rever

Fundamentos: funções puras, imutabilidade e avaliação lazy.

Tipos, assinaturas, aplicação de funções e polimorfismo.

Listas, tuplos, padrões, guardas e definições locais.

Recursão sobre números e listas; acumuladores.

Treino: 8 escolhas múltiplas e 7 exercícios de resposta escrita.

Fundamentos de Haskell

Função pura: os mesmos argumentos produzem o mesmo resultado, sem efeitos laterais observáveis.

Imutabilidade: construímos novos valores; não alteramos os existentes.

Transparência referencial: podemos substituir uma expressão pura pelo seu valor sem alterar o significado do programa.

Avaliação lazy: uma expressão é avaliada quando é necessária.

Tipagem estática: os tipos são verificados antes da execução.

Valores, tipos e inferência

- Bool: True, False
- Char: 'a'
- String: "abc" — String é um sinónimo de [Char].
- Int: inteiros de tamanho limitado; Integer: precisão arbitrária.
- Float e Double: números de vírgula flutuante aproximados.
- :t consulta o tipo.
- :: escreve uma assinatura ou anotação. — Haskell infere tipos mesmo quando não escrevemos a assinatura.

Um literal como 3 pode ter vários tipos numéricos; não é sempre Int.

Ler uma assinatura de tipos

```
pertence :: Eq a => a -> [a] -> Bool
```

Como se lê?

a: tipo do valor procurado; [a]: lista de valores desse mesmo tipo.
Bool: resultado lógico — True ou False.
Eq a => exige que seja possível comparar valores com ==.
A assinatura descreve os argumentos e o resultado, não o algoritmo.

Polimorfismo e restrições

SEM RESTRIÇÃO

```
troca :: (a,b) -> (b,a)
troca (x,y) = (y,x)
```

Os tipos a e b podem
ser iguais ou diferentes.

COM RESTRIÇÃO

```
dobro :: Num a => a -> a
dobro x = 2*x
```

Num a permite usar
operações numéricas.

Eq: igualdade · Ord: ordenação · Num: operações numéricas

Dois argumentos ou um par?

ARGUMENTOS SEPARADOS

```
soma :: Int -> Int -> Int
soma x y = x+y

soma 2 3  -- 5
soma 2 :: Int -> Int
```

UM ARGUMENTO: UM PAR

```
somaPar :: (Int,Int) -> Int
somaPar (x,y) = x+y

somaPar (2,3)  -- 5
```

Int -> Int -> Int significa Int -> (Int -> Int)

Aplicação de funções e operadores

```
quadrado :: Int -> Int
quadrado x = x*x

quadrado 2 + 3      -- 7
quadrado (2 + 3)    -- 25
```

Parênteses e precedência

f x + y significa (f x) + y.
(+) 2 3 e 2 + 3 representam a mesma aplicação.
7 `div` 2 = 3; 7 `mod` 2 = 1; 7 / 2 = 3.5.
Para um argumento negativo, escreve: quadrado (-3).

Listas e tuplos

LISTA: ELEMENTOS DO MESMO TIPO

[True, False] :: [Bool]
["Ana", "Rui"] :: [String]
[] :: [a]

O comprimento pode variar.

TUPLO: POSIÇÕES COM TIPOS FIXOS

("Ana", 20) :: (String,Int)
(True, 'a') :: (Bool,Char)

O número de posições faz parte da estrutura do tipo.

"abc" :: [Char] | ["abc"] :: [String]

Construir e decompor listas

```
(:)  :: a -> [a] -> [a]
(++) :: [a] -> [a] -> [a]

1 : [2,3]       -- [1,2,3]
[1,2] ++ [3]    -- [1,2,3]
```

O padrão (x:xs)

Para [10,20,30]: x = 10 e xs = [20,30].
x é um elemento; xs continua a ser uma lista.
[] é vazia; [x] tem exatamente um elemento; (x:xs) não é vazia.
1 ++ [2,3] é um erro de tipos: o primeiro argumento não é uma lista.

Pattern matching e ordem das cláusulas

```
f :: (Int,Char,Int) -> Int
f (y,'a',x) = y+x
f (z,'b',x) = z*x
f (x,_,_)   = x
```

A primeira cláusula compatível é escolhida

f (3,'a',5) = 8; f (9,'B',0) = 9.
'B' e 'b' são caracteres diferentes.
Se o caso geral vier primeiro, esconde os casos seguintes.
_ ignora um valor; não cria uma variável que possamos usar.

Guardas e if / then / else

```
sinal :: Int -> Int
sinal n
  | n > 0     = 1
  | n == 0    = 0
  | otherwise = -1
```

Condições sobre valores

As guardas são testadas de cima para baixo e têm tipo Bool.
otherwise equivale a True.
if n >= 0 then n else -n devolve um valor.
Os ramos then e else têm de ter tipos compatíveis.

Definições locais: where e let / in

WHERE

```
media :: Double -> Double
      -> Double
media x y = total / 2
  where
    total = x+y
```

LET / IN

```
media :: Double -> Double
      -> Double
media x y =
  let total = x+y
  in total / 2
```

As duas versões recebem dois Double e devolvem a média

case e funções parciais

```
vazia :: [a] -> Bool
vazia xs = case xs of
  []    -> True
  (_:_) -> False
```

Cobrir os casos possíveis

case escolhe uma alternativa pela estrutura do valor.
head e tail exigem listas não vazias: head [] e tail [] falham.
Um programa pode ter tipos corretos e ainda falhar em execução.
Define o domínio e testa os casos limite.

Recursão sobre números

```
fatorial :: Integer -> Integer
fatorial 0 = 1
fatorial n = n * fatorial (n-1)
```

Domínio considerado: n >= 0

Caso base: fatorial 0 = 1.
Redução do problema: n passa para n-1.
fatorial 3 → 3 * fatorial 2 → 3 * 2 * fatorial 1
→ 3 * 2 * 1 * fatorial 0 → 6
Para n negativo, esta definição não atinge o caso base.

Recursão sobre listas

SOMAR VALORES

```
soma :: Num a => [a] -> a
soma [] = 0
soma (x:xs) = x + soma xs
```

CONTAR ELEMENTOS

```
comprimento :: [a] -> Int
comprimento [] = 0
comprimento (_:xs) =
  1 + comprimento xs
```

Em ambos os casos, a chamada recursiva recebe a cauda xs

Escolher o caso base

Soma: soma [] = 0; produto: produto [] = 1.

Construção de listas: o resultado de [] é normalmente [].

Existe algum elemento que verifica a condição?
Na lista vazia, a resposta é False.

Todos os elementos verificam a condição?
Na lista vazia, a resposta é True: não existe um contraexemplo.

O caso base tem de corresponder ao significado da função.

Construção recursiva e paragem antecipada

```
pertence :: Eq a => a -> [a] -> Bool
pertence _ [] = False
pertence v (x:xs)
  | v == x    = True
  | otherwise = pertence v xs
```

O resultado pode ficar decidido antes do fim

Ao encontrar v, devolvemos True sem percorrer o resto.
Para construir uma lista, usamos x : resultadoRecursivo.
Para contar, usamos 1 + resultadoRecursivo quando a condição vale.
Não confundir uma contagem com a soma dos valores.

Acumuladores

```
reverseAcc :: [a] -> [a]
reverseAcc xs = aux xs []
  where
    aux [] acc = acc
    aux (x:xs) acc = aux xs (x:acc)
```

O acumulador transporta o resultado parcial

aux [1,2,3] [] → aux [2,3] [1] → aux [3] [2,1]
→ aux [] [3,2,1] → [3,2,1]
A lista por tratar diminui; acc contém o prefixo já invertido.
Cada chamada recebe um novo valor de acc: não há mutação.

Inverter uma lista: correção e eficiência

CONCATENAR NO FIM

```
inverte :: [a] -> [a]
inverte [] = []
inverte (x:xs) =
  inverte xs ++ [x]
```

Custo total: $O(n^2)$.

ACRESCENTAR À CABEÇA

```
aux [] acc = acc
aux (x:xs) acc =
  aux xs (x:acc)
```

Começar com acc = [].
Custo total: O(n).

(++) percorre a lista da esquerda; (:) acrescenta à cabeça

Recursão com várias chamadas

```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

Fibonacci: domínio n >= 0

fib 4 → fib 3 + fib 2 → 2 + 1 → 3.
As duas chamadas aproximam-se de um caso base.
A versão direta repete cálculos, como fib 2.
Convenção usada aqui: fib 0 = 0 e fib 1 = 1.

QuickSort: dividir, resolver e combinar

Escolher o primeiro elemento como pivô.

Separar a cauda em valores menores ou iguais e valores maiores.

Ordenar recursivamente as duas partes e colocar o pivô entre elas.

[3,1,2,3] → ordenar [1,2,3] ++ [3] ++ ordenar []
→ [1,2,3,3]

Caso base: []. Cada parte vem da cauda e é menor que a entrada.

Como resolver na frequência

Escolha múltipla: verifica primeiro os tipos e só depois o resultado.

Padrões e guardas: segue a ordem escrita, de cima para baixo.

Código: escreve a assinatura, o caso base e o caso recursivo.

Justifica o que diminui e como combinas o resultado recursivo.

Testa: caso vazio, um elemento, caso normal e valores de fronteira.

As perguntas seguintes são treino; escolhe uma única opção.

Escolha múltipla 1 — funções puras

Qual é a afirmação correta?

A. Uma função pura altera os argumentos recebidos.

B. Uma função pura tem de usar recursão.

C. Com os mesmos argumentos, uma função pura produz o mesmo resultado e não tem efeitos laterais.

D. Lazy significa que os tipos só são verificados em execução.

Escolhe uma opção e justifica.

Resolução 1 — opção C

C descreve o comportamento de uma função pura.

A é falsa: os valores são imutáveis.
B é falsa: dobro x = 2*x é pura e não é recursiva.
D é falsa: lazy refere-se à avaliação; a tipagem é estática.

Exemplo: substituir dobro 4 por 8 preserva o significado numa expressão pura.

Escolha múltipla 2 — String ou Char?

Que expressão tem um tipo válido?

A. [tail "abc", head "abc"]

B. (tail "abc", head "abc")

C. True + 1

D. [True, 'a']

Indica também o valor e o tipo da opção escolhida.

Resolução 2 — opção B

```
(tail "abc", head "abc")
→ ("bc", 'a')

("bc", 'a') :: (String, Char)
```

O tuplo admite componentes de tipos diferentes

tail "abc" tem tipo String; head "abc" tem tipo Char.
A é inválida: uma lista não mistura String e Char.
C tenta somar um Bool; D mistura Bool e Char numa lista.

Escolha múltipla 3 — inferir um tipo

Escolhe o tipo mais geral de troca.

troca (x,y) = (y,x)

A. (a,a) -> (a,a)

B. a -> b -> (b,a)

C. Eq a => (a,a) -> (a,a)

D. (a,b) -> (b,a)

A função compara valores ou apenas troca as suas posições?

Resolução 3 — opção D

```
troca :: (a,b) -> (b,a)
troca (x,y) = (y,x)

troca (True, 'a')  -- ('a', True)
```

Um par entra; um par com as posições trocadas sai

x :: a e y :: b; por isso (y,x) :: (b,a).
A e C restringem desnecessariamente os dois elementos ao mesmo tipo.
C acrescenta Eq sem haver comparação.
B descreve dois argumentos separados, não um único par.

Escolha múltipla 4 — currying

```
soma :: Int -> Int -> Int
soma x y = x+y
```

Qual é o tipo de soma 2?

A. Int -> Int

B. Int

C. (Int,Int) -> Int

D. É um erro: falta obrigatoriamente o segundo argumento.

Lê Int -> Int -> Int com as setas associadas à direita.

Resolução 4 — opção A

```
soma :: Int -> (Int -> Int)
soma 2 :: Int -> Int

(soma 2) 5 → 2 + 5 → 7
```

Aplicação parcial

Ao fornecer 2, fica uma função à espera de outro Int.
O resultado final é Int apenas depois do segundo argumento.
C corresponde a uma função que recebe um par.
D é falsa: dar menos argumentos pode produzir outra função.

Escolha múltipla 5 — construção de listas

Qual é o resultado de 1 : [2] ++ [3,4]?

A. [[1,2],[3,4]]

B. [1,2,[3,4]]

C. [1,2,3,4]

D. Erro de tipos: (:) exige duas listas.

Distingue o argumento de (:) dos argumentos de (++).

Resolução 5 — opção C

```
1 : ([2] ++ [3,4])
→ 1 : [2,3,4]
→ [1,2,3,4]
```

Construção e concatenação

(:) recebe um elemento e uma lista.
(++) recebe duas listas com elementos do mesmo tipo.
A cria uma lista de listas; B mistura números com uma lista.
D troca os papéis dos operadores.

Escolha múltipla 6 — ordem dos padrões

```
f :: (Int,Char,Int) -> Int
f (y,'a',x) = y+x
f (z,'b',x) = z*x
f (x,_,_)   = x
```

Quanto vale f (5,'b',4)?

A. 9 B. 20 C. 5 D. 4

Procura a primeira cláusula cujo padrão corresponde.

Resolução 6 — opção B

```
f (5,'b',4)
→ 5 * 4
→ 20
```

O segundo padrão é o primeiro compatível

A primeira cláusula exige 'a', por isso é ignorada.
Na segunda: z = 5 e x = 4. O resultado é z*x.
O caso geral não é usado, apesar de também corresponder.
Se viesse em primeiro lugar, o resultado seria 5.

Escolha múltipla 7 — terminação

```
soma [] = 0
soma (x:xs) = x + soma (x:xs)
```

O que acontece ao tentar obter soma [1,2]?

A. Devolve 3.
B. Devolve 0.
C. É um erro de tipos.
D. Não produz resultado: a chamada repete a mesma lista.

Ter um caso base é suficiente para garantir a terminação?

Resolução 7 — opção D

```
soma :: Num a => [a] -> a
soma [] = 0
soma (x:xs) = x + soma xs
```

Correção: chamar a função sobre xs

Na versão errada, a lista recebida nunca diminui.
O caso [] existe, mas não é alcançado para [1,2].
Com a correção: soma [1,2] → 1 + soma [2]
→ 1 + 2 + soma [] → 3.

Escolha múltipla 8 — acumulador

```
aux [] acc = acc
aux (x:xs) acc = aux xs (x:acc)
```

Partindo de aux [1,2,3] [], qual é a chamada
após processar os dois primeiros elementos?

A. aux [3] [2,1]
B. aux [3] [1,2]
C. aux [] [1,2,3]
D. aux [2,3] [1]

Em que extremidade de acc é colocado cada elemento?

Resolução 8 — opção A

```
aux [1,2,3] []
→ aux [2,3] [1]
→ aux [3] [2,1]
→ aux [] [3,2,1]
→ [3,2,1]
```

O acumulador guarda o prefixo já invertido

O operador (:) coloca o elemento à cabeça.
B mantém a ordem original; C apresenta o fim com ordem errada.
D é a chamada após apenas um elemento.
No caso base, devolvemos acc, não [].

Código 1 — potência inteira

Implementa a potência por recursão, sem usar (^).

```
potencia :: Integer -> Int -> Integer
```

Exemplos esperados

```
potencia 2 4   == 16
potencia 5 0   == 1
potencia 0 3   == 0
```

Assume expoente n >= 0; usa potencia x 0 = 1.

Resolução — potência inteira

```
potencia :: Integer -> Int -> Integer
potencia _ 0 = 1
potencia x n = x * potencia x (n-1)
```

Base, redução e resultado

Recebe uma base Integer e um expoente Int; devolve Integer.
Caso base: expoente 0. O expoente diminui em cada chamada.
potencia 2 3 → 2 * potencia 2 2 → 2 * 2 * potencia 2 1
→ 2 * 2 * 2 * potencia 2 0 → 8.
Expoentes negativos não são admitidos neste enunciado.

Código 2 — tempos válidos numa corrida

Conta por recursão os pilotos com tempo inferior a 5 minutos.

```
countValidTimes :: [(String,Int)] -> Int
```

Exemplos esperados

```
countValidTimes [("Ana",299),("Rui",300)] == 1
countValidTimes [("Eva",301)]             == 0
countValidTimes []                        == 0
```

Tempos não negativos, em segundos. Não uses funções auxiliares.

Resolução — tempos válidos numa corrida

```
countValidTimes :: [(String,Int)] -> Int
countValidTimes [] = 0
countValidTimes ((_,t):xs)
  | t < 300   = 1 + countValidTimes xs
  | otherwise = countValidTimes xs
```

5 minutos = 300 segundos; o limite é estrito

Recebe pares (nome,tempo) e devolve uma contagem Int.
O padrão ((_,t):xs) ignora o nome e extrai o tempo do primeiro par.
299 conta; 300 e 301 não contam. Não somamos os tempos.
Em ambos os ramos, a chamada recebe xs. A lista vazia dá 0.

Código 3 — remover a primeira ocorrência

Remove apenas a primeira ocorrência, preservando a ordem.

```
removePrimeiro :: Eq a => a -> [a] -> [a]
```

Exemplos esperados

```
removePrimeiro 3 [1,3,2,3] == [1,2,3]
removePrimeiro 9 [1,2]     == [1,2]
removePrimeiro 3 []        == []
```

Usa recursão e padrões. Não removas as ocorrências seguintes.

Resolução — remover a primeira ocorrência

```
removePrimeiro :: Eq a => a -> [a] -> [a]
removePrimeiro _ [] = []
removePrimeiro v (x:xs)
  | v == x    = xs
  | otherwise = x : removePrimeiro v xs
```

Eq a permite comparar v com x

Recebe um valor e uma lista do mesmo tipo; devolve uma lista.
Se encontrar v, devolve xs: para e preserva as ocorrências seguintes.
Se não encontrar, conserva x e continua na cauda.
removePrimeiro 3 [1,3,2,3] → 1 : [2,3] → [1,2,3].

Resposta escrita 4 — despesas por dia

Define totalDia e gastosMes usando recursão em gastosMes.
Podes usar sum para somar os valores de um dia.

totalDia :: (Int,[Double]) -> (Int,Double)
gastosMes :: [(Int,[Double])] -> [(Int,Double)]

Cada par contém o dia do mês e as despesas desse dia.
Preserva os dias e a ordem da lista, mesmo se faltarem dias.

gastosMes [(1,[10,2]),(3,[]),(5,[4,6])]
→ [(1,12.0),(3,0.0),(5,10.0)]

Escreve as duas funções. Indica também o resultado para [].

Resolução — despesas por dia

```
totalDia :: (Int,[Double]) -> (Int,Double)
totalDia (dia, valores) = (dia, sum valores)

gastosMes :: [(Int,[Double])] -> [(Int,Double)]
gastosMes [] = []
gastosMes (d:ds) = totalDia d : gastosMes ds
```

Transformar cada par; conservar a estrutura da lista

totalDia recebe um par e devolve o dia com o total das despesas.
gastosMes recebe e devolve listas de pares; o segundo campo muda.
Para (3,[]): sum [] = 0, logo totalDia (3,[]) = (3,0.0).
Caso base: gastosMes [] = []. Em cada passo, ds é mais pequena.

Resposta escrita 5 — classificação ordenada

Define uma função recursiva que verifica a ordem dos tempos.
Os tempos estão em segundos; empates são permitidos.

```
isRankingValid :: [(String,Int)] -> Bool

isRankingValid [("Ana",270),("Rui",270),("Eva",310)]
→ True
isRankingValid [("Ana",270),("Rui",310),("Eva",290)]
→ False
```

Uma lista vazia ou com um único piloto é considerada ordenada.

Compara vizinhos. Explica que lista passas à chamada recursiva.

Resolução — classificação ordenada

```
isRankingValid :: [(String,Int)] -> Bool
isRankingValid [] = True
isRankingValid [_] = True
isRankingValid ((_,t1):(n2,t2):xs) =
  t1 <= t2 && isRankingValid ((n2,t2):xs)
```

O segundo piloto tem de participar na comparação seguinte

O resultado é Bool: todos os pares de tempos vizinhos respeitam <=.
A recursão conserva (n2,t2) e avança apenas um elemento.
Para 270,310,290: 270 <= 310 é True; 310 <= 290 é False.
Usar apenas xs saltaria essa segunda comparação. [] e [_] dão True.

Resposta escrita 6 — atletas em comum

Devolve os nomes da primeira prova que também estão na segunda.
Preserva a ordem e as repetições da primeira lista.

commonAthletes :: [String] -> [String] -> [String]

commonAthletes ["Ana","Rui","Ana"] ["Rui","Ana"]
→ ["Ana","Rui","Ana"]
commonAthletes ["Ana","Eva"] ["Rui"] → []

Podes reutilizar a função recursiva pertence já revista:
pertence :: Eq a => a -> [a] -> Bool

Usa recursão explícita. Testa também uma das listas vazia.

Resolução — atletas em comum

```
commonAthletes :: [String] -> [String] -> [String]
commonAthletes [] _ = []
commonAthletes (x:xs) ys
  | pertence x ys = x : commonAthletes xs ys
  | otherwise     = commonAthletes xs ys
```

Percorrer a primeira lista; pesquisar sempre na segunda

Recebe duas listas de nomes e devolve uma lista de nomes.
Se x pertence a ys, conserva-o; caso contrário, ignora-o.
Só xs diminui neste percurso: ys permanece a lista de referência.
Com ys = [], nenhuma pesquisa tem sucesso; o resultado é [].

Resposta escrita 7 — detetar uma suposição

As listas têm os mesmos pilotos, mas podem ter ordens diferentes.

```
best [] [] = []
best ((n,t):xs) ((_,u):ys) =
  (n, min t u) : best xs ys

xs = [("Ana",280),("Rui",250)]
ys = [("Rui",240),("Ana",270)]
```

1. Que resultado produz best xs ys? É o resultado correto?
2. Que condição adicional tornaria esta estratégia válida?
3. Explica como resolver o problema para ordens diferentes.

min escolhe o menor tempo. Assume nomes únicos em cada prova.

Resolução — comparar pilotos, não posições

1. A função produz [("Ana",240),("Rui",250)].
   Está errado: atribui à Ana o tempo 240, que é do Rui.
   O resultado correto é [("Ana",270),("Rui",240)].

2. A comparação por posição exige os mesmos pilotos,
   pela mesma ordem e com listas do mesmo comprimento.

3. Para cada (nome,tempo) da primeira lista, pesquisar
   recursivamente esse nome na segunda e comparar os tempos.
   Depois continuar na cauda da primeira lista.

Mesmo comprimento e mesmos nomes não garantem o alinhamento.